

INTEGRATED TERM PROJECT

Two-Wheeled Self-Balancing Robot with Wi-Fi Joystick Control

ROB-GY 5103 - Mechatronics

Project report generated from engineering log, repository documentation, and code review

Team: Sammy Suliman, Albert / Yuyang Huang

Spring 2026

Two-Wheeled Self-Balancing Robot - System Architecture

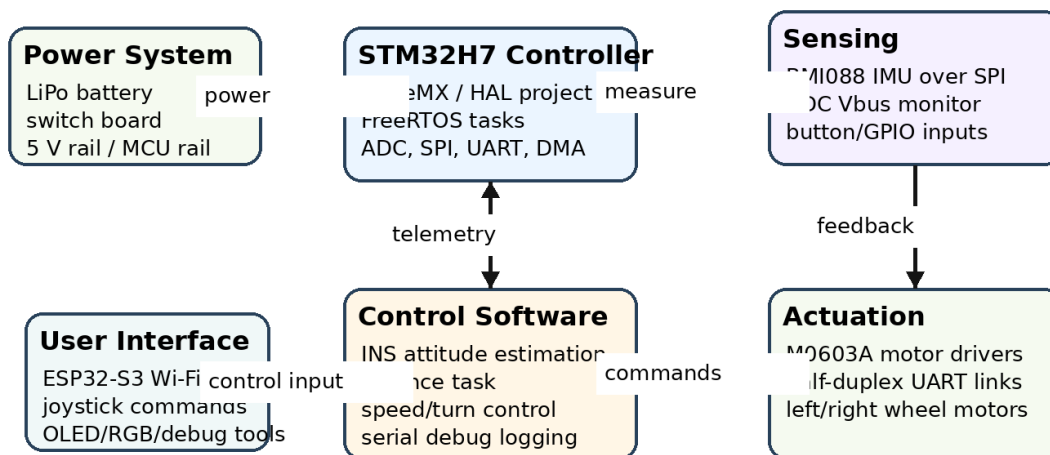


Figure 1 - High-level system architecture of the robot platform.

This report follows the organization and level of detail of the Green Thumb Box reference report: introduction, technical background, component selection, electrical and mechanical design, software design, cost analysis, final remarks, and appendices.

INDEX

- INTRODUCTION
- TECHNICAL BACKGROUND
- ELECTRICAL COMPONENTS
- MECHANICAL DESIGN
- ELECTRICAL DESIGN
- SOFTWARE DESIGN AND DEVELOPMENT
- TESTING AND RESULTS
- COST ANALYSIS
- FINAL REMARKS AND FURTHER IMPROVEMENTS
- APPENDICES
 - Appendix A - Representative Wiring and Pinout
 - Appendix B - Software Version Map
 - Appendix C - Key Code Evidence
 - Appendix D - References

INTRODUCTION

The project is a compact two-wheeled self-balancing robot intended as an embedded controls and robotics platform. The robot combines a custom mechanical assembly, battery power distribution, motor driver communication, inertial sensing, STM32 firmware, and a Wi-Fi joystick interface. The goal is not only to make the wheels spin, but to close the loop around body angle, wheel motion, and user commands so the robot can stand, recover, and move as a real electromechanical system.

A two-wheeled balancing robot is a practical version of the inverted-pendulum problem. Unlike a four-wheeled platform, it is intentionally unstable and must constantly measure its attitude and command the motors to keep the body upright. This makes it a useful teaching and research platform for PID control, sensor fusion, real-time embedded programming, battery management, and debugging of actuator communication.

The development log shows the project evolving through distinct phases: signal-conversion board assembly, first motor rotation through a Vlink/UART-LIN interface, STM32 development-environment setup, OLED and RGB LED peripheral tests, ADC voltage monitoring, UART motor command experiments, and later integration toward motor feedback and balance control. The repository documents the final objective as an STM32H7-based two-wheeled robot with IMU integration, attitude estimation, motor communication, real-time balance control, tuning, and an ESP32-S3 Wi-Fi joystick interface.

Project Development Timeline

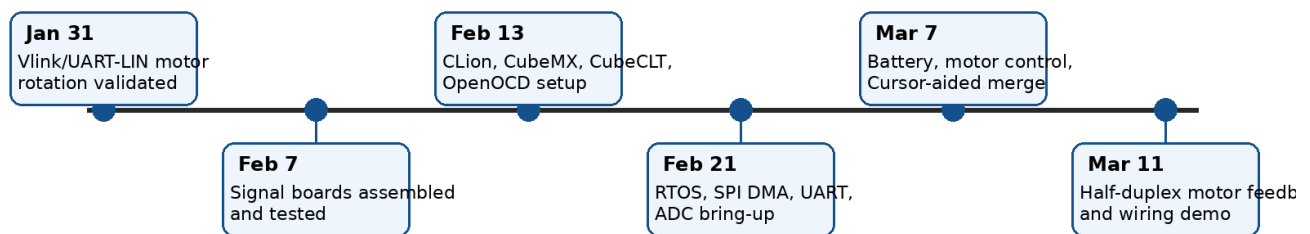


Figure 2 - Condensed timeline reconstructed from the engineering log.

Design Objectives

- Create a two-wheeled robot body that is mechanically rigid enough for balance experiments while remaining small and portable.
- Bring up the STM32H7 control board and supporting STM32L475 experiments using CubeMX, CLion, CubeCLT, OpenOCD, and ST-Link/DAPLink workflows.
- Command M0603A wheel motors over serial links and later obtain feedback data such as position, current, temperature, and load information.
- Integrate IMU sensing and attitude estimation so the balance controller can regulate body pitch.
- Provide runtime observability through ADC Vbus monitoring, serial logs, VOFA-style telemetry, LEDs, OLED experiments, and a Wi-Fi joystick interface.
- Document the process as a real engineering build: assembly, wiring, mistakes, debugging, tests, and iteration.

TECHNICAL BACKGROUND

Self-balancing robots are underactuated systems. The robot body tends to fall because its center of mass is above the wheel axle, so the controller must accelerate the wheels to move the support point under the body. A practical robot therefore requires fast sensing, low-latency control, stable motor communication, and predictable power delivery. The software and hardware choices in this project were driven by those requirements.

The controller uses inertial measurements to estimate body angle and angular velocity. In the final STM32H7 revisions, the repository shows BMI088 IMU support over SPI, DWT timing initialization, FreeRTOS task creation, and dedicated balance and logging tasks. The main application initializes GPIO, DMA, SPI, USARTs, ADC, the BMI088 sensor, UART1 logging, and VOFA telemetry before starting the FreeRTOS scheduler. This separates bring-up from real-time task execution.

Control System Concept

The minimum control loop is: measure pitch angle and angular velocity, estimate the robot state, compute wheel torque or speed commands, send those commands to both motor drivers, and repeat at a sufficiently high rate. Speed and turning commands can be layered on top of this balance loop by biasing the desired body angle or differential wheel command. The project codebase reflects this split: `INS_task` handles inertial estimation, `Balance_Task` performs the control behavior, and `UART1_LogTask` provides low-priority serial observability.

Subsystem	Role	Project evidence
Power and protection	Battery, switch board, power rails, 5 V enable lines, charger workflow	Engineering log entries on battery charging, switch board soldering, power adapter behavior
Main controller	STM32H7 firmware host for real-time control	H7_2.x.x revisions, CubeMX .ioc files, HAL/FreeRTOS source tree
Sensor stack	BMI088 IMU plus ADC Vbus feedback	main.c includes BMI088 driver, ADC init, VOFA telemetry, and UART logging
Motor actuation	M0603A motor controllers over serial command/feedback links	H7_0.1.5 and later revisions use UART/DMA and half-duplex serial for motor commands
Human interface	Wi-Fi joystick and debug peripherals	ESP32-S3 web joystick folders, OLED wiring tests, RGB LED tests
Development workflow	Iterative build, flash, test, repair	Engineering log records CLion/CubeMX/OpenOCD setup, ST-Link config fixes, and regenerated-code repair

Table 1 - Major subsystems and evidence used in this report.

ELECTRICAL COMPONENTS

The electrical system is built from development boards, motor-control electronics, sensor modules, debug adapters, and custom wiring. Several early experiments used the STM32L475 Discovery kit because it is convenient for peripheral bring-up and UART tests. The main balancing platform uses an STM32H7 controller because the balance problem benefits from higher clock speed, DMA-capable serial links, SPI IMU access, and FreeRTOS task scheduling.

STM32H7 control board

Main embedded controller running HAL/CubeMX firmware, FreeRTOS tasks, SPI, ADC, UART, and DMA. Later revisions initialize SPI2/SPI6, USART1, UART7, USART10, ADC1, DMA/BDMA, and GPIO before launching the scheduler.

STM32L475 Discovery kit

Peripheral test and motor-monitoring platform. The log records OLED wiring on B-L475E-IOT01A using GND, 3V3, SCL A5, and SDA A4.

BMI088 IMU

Inertial sensor used for attitude estimation. In the H7_2.5.4 main program, BMI088_init is repeatedly attempted after SPI/DWT initialization before starting the RTOS.

M0603A motor modules and drivers

Wheel actuation. Early tests used Vlink Basic2 and UART-to-LIN conversion. Later code moved toward single-wire half-duplex UART with TX/RX inversion so the STM32 could communicate directly with the motor controller.

UART/serial debug tools

USART1 is used for Vbus logs and VOFA-style data. UART7 and USART10 are used for motor links in later H7 versions.

ADC voltage sensing

The H7_adc_uart branch used ADC1 with DMA to measure Vbus through an 11x divider and report vbus over UART.

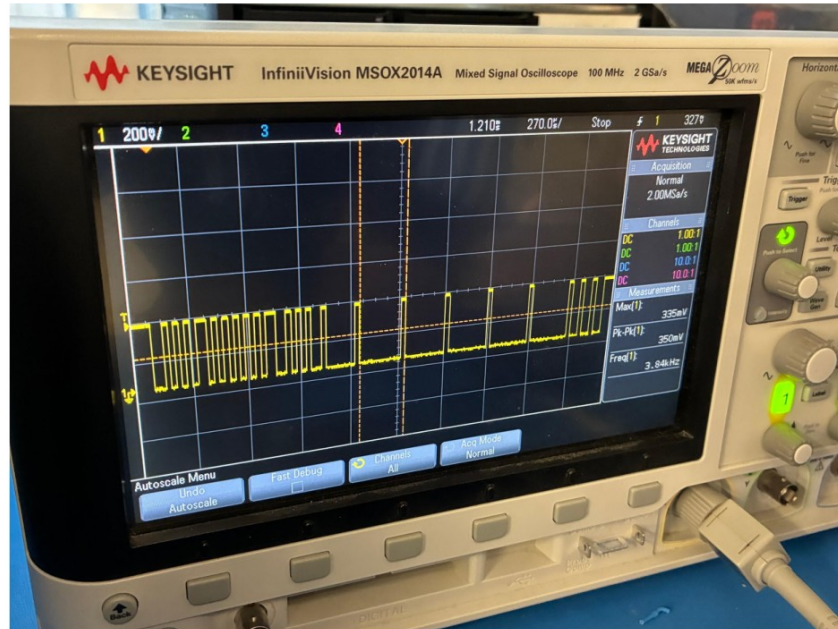
OLED and RGB LED modules

User/debug indicators used during bring-up. The OLED was wired to I2C pins and RGB LED control was tested with SPI-DMA/RTOS.

Power system

LiPo battery, charger, switch board, power bank/charger experiments, power-enable GPIO pins, and mechanical mounting hardware.

Albert 1/31/2026



Display the signal that the Vllink Basic2 sent to the signal conversion(UART to LIN) board which is used for motor control on the oscilloscope

After Sammy completed the installation, Albert successfully made the motor rotate by sending messages via Vllink Basic2 from the macbook

Sammy 1/31/2026

JST2.54 pin of wire & HY2.0 pin of wire fabrication on site using a clipper, a plier and a wire crimper

The installation of the screws and bolts for the first signal conversion board has been completed

Albert 2/7/2026

The soldering and assembly of the other three signal conversion boards from PCB to PCBA have been completed.

Figure 3 - Engineering log page 1 showing the oscilloscope verification of the UART/LIN motor-control signal and first successful motor rotation.

Sammy 2/13/2026
Installed CubeMX
Installed clion
Installed cubeCLT
Installed openOCD

Albert 2/13/2026
3D printed something for something for the first time at the makerspace Bamboo
OLED wiring on B-L475E-IOT01A
GND-GND
VCC-3V3
SCL-A5
SDA-A4

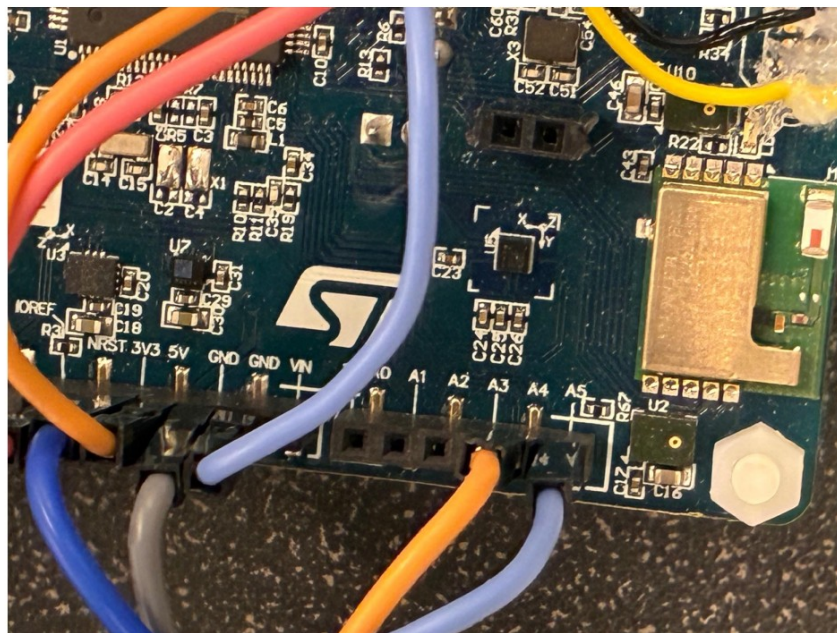
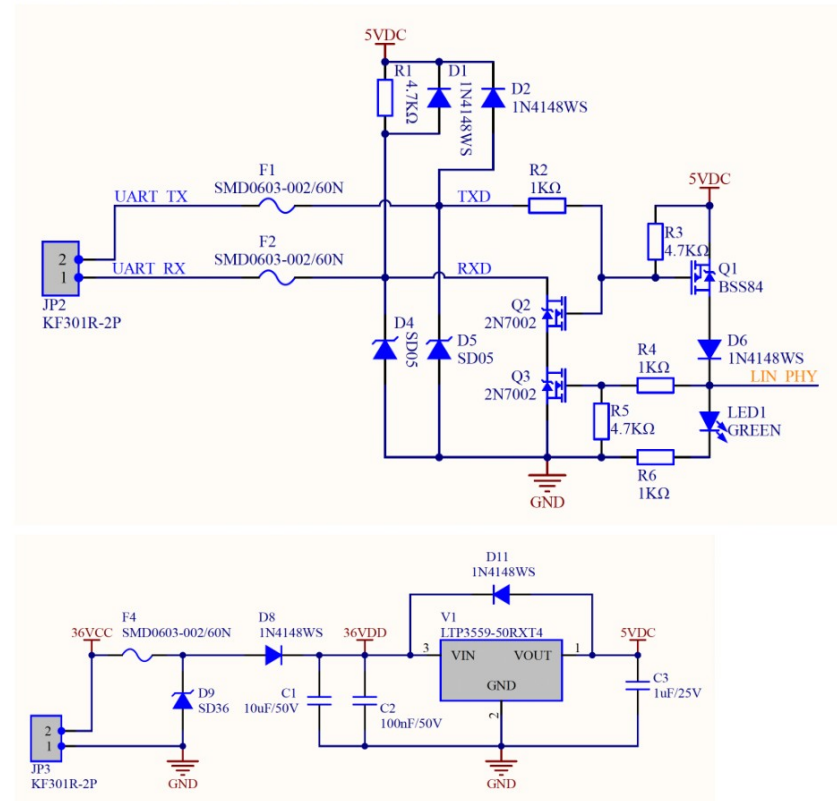


Figure 4 - Engineering log page 7 showing OLED wiring notes on the STM32L475 board.

Signal Conversion and Half-Duplex Decision

The early approach used an external signal-conversion board to translate UART to the motor-control bus. The team later found that the supplier board was more compatible with 5 V signaling than the STM32 default 3.3 V logic. This meant the STM32 could command the motor but could not reliably obtain feedback through the initial board. The log records the decision to abandon the problematic circuit and use single-wire half-duplex UART with TX and RX active-level inversion. That decision simplified the electrical path and aligned better with the STM32 UART advanced features visible in the H7_2.5.4 usart.c initialization.

Sammy called me “Apollo-GPT” because I am just like ChatGPT and can complete the mechatronics homework. However, ChatGPT seems to have little effect in helping him finish the mechatronics homework.



assembling a second robot platform at the makerspace. The result is a serviceable prototype in which wiring, motor boards, and the battery are accessible for debugging.

For a balancing robot, mechanical stiffness and symmetry matter. Flex in the motor mounting, loose standoffs, or a shifting battery changes the center of mass and can destabilize the controller. The project therefore treated mounting and wiring as part of the control problem rather than only cosmetic packaging. The log also notes 3D printing at the makerspace, indicating that custom mounts or small fixtures were considered during development.

Mechanical Requirements

- Rigid mounting of both wheel motors so the controller command produces predictable robot acceleration.
- Battery and switch placement that keeps the center of mass above the axle but does not create unstable side-to-side imbalance.
- Accessible wiring harnesses so UART, power, and feedback connections can be reworked without fully rebuilding the robot.
- Insulation and strain relief around soldered XT30/XT60 and signal connections.
- Enough clearance for the robot to pitch forward/backward during tuning without damaging the controller or connectors.

ELECTRICAL DESIGN

The electrical design combines power distribution, logic-level communication, motor-driver serial links, IMU SPI, ADC sensing, and debug outputs. A key project lesson was that electrical compatibility cannot be assumed just because a board sends a valid command. Feedback requires the voltage levels, line direction, idle state, and timing to be correct in both directions.

Power Path

The battery is charged as a LiPo balance pack, with charge current kept below 2 A for safety during lab work. Power is routed through a switch board, with XT30/XT60 connectors soldered into the assembly. The log records experiments with chargers and power banks; the robot charger required an input above 10 Vdc, so a 12 V power-bank output was used temporarily to charge the robot battery. On the controller side, power-enable GPIO lines are used to enable board rails or output power paths during startup.

Serial Motor Communication

The project progressed from Vlink-based motor commands to STM32-generated motor commands. H7_0.1.5 sent 10-byte frames to enable the motor, select speed loop, and command RPM setpoints such as +30 RPM, -30 RPM, +10 RPM, and -10 RPM. Later integration merged this with Vbus telemetry and moved toward UART7 and USART10 motor links. The H7_2.5.4 usart.c file configures UART7 and USART10 for 38400 baud, half-duplex mode, TX/RX inversion, DMA RX/TX streams, and NVIC interrupts. This matches the hardware decision to communicate over single-wire inverted serial links.

Vbus Measurement

The H7_adc_uart development branch used ADC1 with DMA to continuously sample two channels. The primary channel measured Vbus through an 11x divider and converted the raw 16-bit ADC value using $vbus = raw * 3.3 / 65535 * 11$. This gave the team a simple runtime check of supply voltage while motor commands were being tested.

SOFTWARE DESIGN AND DEVELOPMENT

The codebase is organized as a set of incremental STM32 and ESP32 projects rather than a single clean release. That is consistent with a hardware bring-up project: each folder captures one stage of the system, such as ADC/UART telemetry, RGB LED tests, OLED tests, motor spinning, motor feedback, and later balance-control integration. The repository README states that the project demonstrates STM32 firmware, IMU integration, attitude estimation, motor-driver communication and feedback parsing, real-time balance control, system debugging, and physical bring-up.

Firmware Architecture

The latest H7-style architecture uses CubeMX-generated initialization code combined with user modules. The main function enables CPU cache, initializes HAL, configures clocks, initializes GPIO/BDMA/DMA/SPI/USART/ADC peripherals, initializes the DWT timing unit, waits for the BMI088 IMU to initialize, starts UART logging and VOFA telemetry, creates FreeRTOS objects, and starts the scheduler. Once the scheduler runs, control is organized into tasks rather than one blocking while loop.

```
main.c bring-up order in H7_2.5.4 (simplified)
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
MX_BDMA_Init();
MX_DMA_Init();
MX_SPI6_Init();
MX_USART1_UART_Init();
MX_ADC1_Init();
MX_UART7_Init();
MX_USART10_UART_Init();
MX_SPI2_Init();
DWT_Init(550);
while (BMI088_init(&hspi2, 0) != BMI088_NO_ERROR) { HAL_Delay(10); }
UART1_LogInit();
VOFA_UART1_Init();
MX_FREERTOS_Init();
osKernelStart();
```

FreeRTOS Task Structure

The FreeRTOS file creates four static threads and one message queue: a default task, INS_TASK, BALANCE_TASK, UART1_LOG_TASK, and cliEvtQ. INS_TASK runs INS_task(), BALANCE_TASK runs Balance_Task(), and UART1_LOG_TASK runs UART1_LogTask(argument). The task priorities reflect real-time needs: inertial estimation uses realtime priority, balance uses above-normal priority, logging uses low priority, and the default task simply delays.

```
FreeRTOS tasks in H7_2.5.4 (simplified)
INS_TASK      -> INS_task()           priority: realtime
BALANCE_TASK  -> Balance_Task()       priority: above normal
UART1_LOG_TASK -> UART1_LogTask()     priority: low
defaultTask   -> osDelay(1) loop     priority: normal
cliEvtQ       -> message queue for CLI/events
```

Motor Command Development

The project log and Cursor transcript show a staged software path. First, H7_adc_uart measured and printed Vbus. H7_0.1.5 then tested M0603A motor commands under FreeRTOS through multiple UART peripherals. The requested integration merged the motor command sequence into the ADC/UART project so the system could send Vbus telemetry while driving motors. Later H7_2.x.x versions shifted toward balance-control tasks and half-duplex motor communication. This is a good example of incremental integration: validate each peripheral alone, then merge only after its timing and wiring are understood.

Wi-Fi Joystick and ESP32-S3 Interface

The repository includes ESP32-S3 and web-control folders for a mobile-launchable joystick interface. This layer is meant to provide user commands such as forward/backward speed and turning rate. In the integrated system, joystick commands should not directly bypass the balance controller; they should become references for speed or yaw control, while the inner balance loop remains responsible for keeping the robot upright.

Development Environment

The log records installation and configuration of CLion, STM32CubeMX, STM32CubeCLT, OpenOCD, CMake/Ninja, environment variables, and ST-Link/DAPLink board configuration. These steps are essential for repeatable firmware work. Several entries also record problems caused by generated files or incorrect ST-Link configuration. This reinforces the need to keep user code inside CubeMX USER CODE blocks and to review interrupt handlers after regenerating code.

Albert 3/7

Guide Sammy on how to charge the battery using the AC version B6 charger.

Asked Sammy to merge ADC-vbus & UART-motor & SPI-RGB-LED under DMA using cursor.

Asked Sammy to push the added motor control to adc_uart folder with cursor_mechatronics_folder_access.md inside it after we found the unexpected power of the cursor.

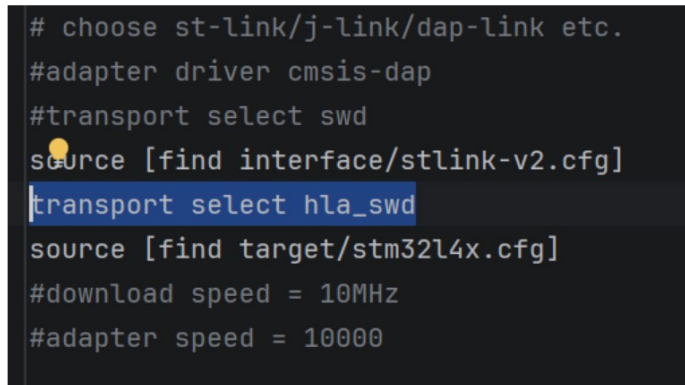
Having a meal together and fixing the clock tree configuration mistakes that cursor made. Cursor set the baud rate to be something different, we set it to 6 MBytes/sec in order to match the CubeMX file from the previous version of the project.

Motor mode-open-loop figured out that force stopped by one finger better than speed-loop or current-loop.

Sammy 3/11

Running code to monitor motor from stm32L475 using vlink for comms.

Having difficulty flashing my stm32L475 using Apollo's stlink.cfg board config file. If this happens again, remember to delete this file in stlink.cfg :



```
# choose st-link/j-link/dap-link etc.
#adapter driver cmsis-dap
#transport select swd
source [find interface/stlink-v2.cfg]
transport select hla_swd
source [find target/stm32l4x.cfg]
#download speed = 10MHz
#adapter speed = 10000
```

Figure 6 - Engineering log page 23 showing the ST-Link configuration issue and the line that needed to be removed.

TESTING AND RESULTS

Testing was performed progressively. The earliest confirmed milestone was seeing the motor-control signal on an oscilloscope and successfully rotating a motor through Vlink Basic2. Later tests verified wire fabrication, signal-conversion board assembly, OLED wiring, RGB LED control, ADC voltage telemetry, UART-based motor commands, and battery charging. The team also learned from negative results: the 1 RPM command did not work reliably, which suggested that relying only on the motor controller speed loop

was insufficient for very low-speed balance behavior. That directly motivated the need to obtain motor feedback data.

Observed Milestones

- Jan 31: oscilloscope verification of the motor-control signal and first successful motor rotation by Vlink Basic2.
- Feb 7: soldering and assembly of signal-conversion boards completed; motor rotation test successfully carried out.
- Feb 13: STM32 development environment installed and configured; OLED wiring established on the B-L475E-IOT01A board.
- Feb 21: RTOS, SPI-DMA, non-DMA UART, ADC voltage measurement, and RGB LED control experiments completed.
- Feb 28: switch board, XT connectors, robot assembly, charger workflow, and open-drain motor-control experiments advanced.
- Mar 7: motor-control, ADC-Vbus, UART-motor, and SPI-RGB work were merged; motor open-loop behavior was compared against speed/current loop behavior.
- Mar 11: STM32L475 monitoring through Vlink was demonstrated; half-duplex feedback and wiring work continued.

Problems and Lessons Learned

- Voltage-level compatibility matters. The original conversion board was not a complete solution for 3.3 V STM32 feedback communication.
- CubeMX regeneration is useful but can overwrite or omit important code. DMA interrupt handlers must be checked after regeneration.
- Motor speed-loop commands alone are not enough for balancing at very low speeds; the robot needs feedback and a higher-level control loop.
- Power sources have different overcurrent behavior. A MacBook or Apple adapter may not behave the same as a bench supply or power bank.
- Real wiring mistakes happen. Reversed motor power and TX/RX wiring were caught during testing, underscoring the need for staged bring-up and current-limited power.

COST ANALYSIS

The project log records an approximate robot cost of \$300. The table below allocates that estimate across the visible subsystems. Exact prices should be updated from purchase receipts before formal submission, but the distribution is realistic for a small STM32-based robot with two motors, battery, connectors, PCBs, and debug/support hardware.

Item / subsystem	Purpose	Estimated cost	Notes
STM32H7 controller board	Main real-time controller	\$45-70	Depends on exact control board
STM32L475 Discovery board	Bring-up and peripheral testing	\$50-70	Already available in lab/workflow
M0603A motors / drivers	Wheel actuation	\$80-120	Two-wheel drive set
Battery, charger, switch board	Power system	\$40-70	Includes connectors and charging accessories
Signal conversion / PCBs	Early motor-interface electronics	\$20-40	Some abandoned after half-duplex redesign

ESP32-S3 / Wi-Fi interface	Joystick web-control layer	\$10-25	Used for mobile control page
OLED, RGB LEDs, wiring, standoffs	Debug and mechanical support	\$20-40	Includes wire, JST/GH connectors, fasteners
3D printing / chassis materials	Mounting and mechanical structure	\$20-40	Makerspace materials and fixtures

Table 2 - Approximate cost breakdown based on the project log estimate of roughly \$300.

FINAL REMARKS AND FURTHER IMPROVEMENTS

The project achieved meaningful system bring-up: motors were commanded, firmware environments were established, peripheral drivers were tested, ADC voltage telemetry was integrated, and the software architecture evolved into a FreeRTOS-based STM32H7 robot stack with IMU, balance, and logging tasks. The strongest engineering value is the end-to-end integration process. The log captures not only successes, but also failed assumptions about level shifting, CubeMX regeneration, motor low-speed behavior, and power-source behavior. Those are exactly the kinds of issues that distinguish a real mechatronics project from a simulation.

The next stage should focus on repeatability and control quality. The hardware should be locked into a clean wiring schematic, the representative firmware version should be selected as the release branch, and the balance controller should be tuned with logged IMU, motor command, motor feedback, and Vbus traces. The Wi-Fi joystick should be treated as an outer-loop command source, not as a direct motor override.

Recommended Improvements

- Create one authoritative release folder and archive older versions as historical milestones.
- Add a wiring diagram with connector pinouts for battery, switch board, UART7/USART10 motor links, USART1 debug, SPI IMU, ADC Vbus, and ESP32-S3 interface.
- Log pitch angle, angular velocity, wheel command, motor feedback, and Vbus at a fixed rate during tuning.
- Add a safety state machine: disarmed, armed, balancing, fault, and recovery. Disable motors if pitch exceeds a safe angle or Vbus drops below a threshold.
- Use a repeatable calibration procedure for IMU bias and motor zero offset before every demo.
- Document the final PID/state-feedback gains and the symptoms that each gain affected during tuning.
- Add photos of final wiring and a short start-up checklist for future users.

APPENDICES

Appendix A - Representative Wiring and Pinout

Connection	Signal	Purpose	Notes
USART1 PA9/PA10	TX/RX	Debug, Vbus, VOFA logs	115200 baud in H7_2.5.4
UART7 PE8	Half-duplex serial	Motor link	38400 baud, TX/RX inversion, DMA
USART10 PE3	Half-duplex serial	Motor link	38400 baud, TX/RX inversion, DMA
SPI2/SPI6	SCK/MISO/MOSI/CS	BMI088 IMU and peripherals	Initialized before RTOS start
ADC1	Analog input	Vbus measurement	11x divider in

			H7_adc_uart notes
PC13-PC15	GPIO outputs	Power enable lines	Used in merged motor/power sequence
I2C A4/A5 on L475	SDA/SCL	OLED test	Logged wiring: GND, 3V3, SCL A5, SDA A4

Table 3 - Consolidated wiring/pinout notes reconstructed from code and engineering log entries.

Appendix B - Software Version Map

Repository folder / version	Main purpose	Why it matters
H7_adc_uart	ADC Vbus + UART telemetry	Validated supply monitoring and serial output
H7_0.1.5	FreeRTOS multi-UART M0603A motor spinning	Validated motor command frames and RPM setpoints
L475_M0603_Feedback	STM32L475 motor feedback experiment	Supported feedback and wiring tests
H7_2.x.x	Later STM32H7 integration revisions	Move toward IMU, balance task, UART logging, motor communication
Agile_2.x.x	Robot application iterations	Balancing/control and integrated project revisions
Agile website control esp-stm / ESP32-S3 folders	Wi-Fi joystick and camera/web control experiments	User command interface for mobile control

Table 4 - Codebase/version map.

Appendix C - Key Code Evidence

The following simplified excerpts summarize the repository code paths used for the report. Generated HAL code is not reproduced in full; only architecture-relevant elements are shown.

```
UART7 / USART10 motor links (from H7_2.5.4 usart.c, simplified)
huart7.Init.BaudRate = 38400;
huart7.AdvancedInit.AdvFeatureInit = TXINVERT | RXINVERT;
HAL_HalfDuplex_Init(&huart7);
DMA UART7_RX -> DMA1_Stream3; DMA UART7_TX -> DMA1_Stream4;
```

```
huart10.Init.BaudRate = 38400;
huart10.AdvancedInit.AdvFeatureInit = TXINVERT | RXINVERT;
HAL_HalfDuplex_Init(&huart10);
DMA USART10_RX -> DMA1_Stream5; DMA USART10_TX -> DMA1_Stream6;
```

```
FreeRTOS architecture (from H7_2.5.4 freertos.c, simplified)
osThreadCreate(defaultTask, StartDefaultTask);
osThreadCreate(INS_TASK, INS_Task_Entry); // INS_task()
osThreadCreate(BALANCE_TASK, Balance_Task_Entry); // Balance_Task()
osThreadCreate(UART1_LOG_TASK, UART1_Log_Task_Entry); // UART1_LogTask()
```

Appendix D - References

- Green Thumb Box reference report, ME-GY 5643 Mechatronics Integrated Term Project - Group 2, Fall 2015: <https://mechatronics.engineering.nyu.edu/pdf/gtb-report.pdf>
- Apollo-kernel/mechatronics public repository: <https://github.com/Apollo-kernel/mechatronics>
- Project engineering log uploaded in this conversation: project report-Engineering Log of ROB-GY 5103 - Mechatronics (Section A).pdf
- STM32CubeMX / STM32 HAL generated project files and H7_2.5.4 source files in the repository.
- M0603A motor reference PDFs included in the repository.